When Data Programming Meets Agent Trajectories: An Empirical Study of Weak Supervision Failure Modes for Progress Stagnation Detection

Anonymous

Abstract

LLM-based agents frequently enter *progress stagnation*—executing surface-diverse but collectively ineffective action sequences during long-horizon tasks. Detecting such stagnation is important for cost control and intervention, yet the concept itself resists clean operationalization: human annotators achieve only moderate agreement (κ=0.52). We formulate stagnation detection as a *window-level* binary classification problem and build a weak-supervision pipeline with four labeling functions (LFs) and a GRU-based probe (ProgressProbe) trained on the resulting noisy labels. Rather than claiming methodological advances, we present a *systematic empirical study* that exposes three categories of failure modes when applying data programming to sequential agent data: (1) LFs exhibit systematic temporal dynamics (leading/lagging indicators) that violate the i.i.d. assumption of standard data programming, yet a temporal label model fails to outperform FlyingSquid ($\Delta=-0.2$pp on the gold set); (2) the strongest LF (`outcome_backpropagation`) leaks trajectory-level outcome information, inflating label quality by 7.4pp; (3) cross-agent transfer degrades severely (F1 drops from 0.91 to 0.61). On the positive side, GRU temporal modeling shows robust performance on noisy labels (+6.9pp over mean-pooling at conf$\geq$1), and provides a 615$\times$ cheaper alternative to LLM-as-judge for continuous monitoring (gold-set F1 0.72 vs. 0.86 for Claude CoT). Our stagnation taxonomy, window-level formulation, and failure-mode characterization provide empirical foundations for future work on agent progress monitoring.

1 Introduction

LLM-based autonomous agents Yang et al. [2024], Wang et al. [2024] increasingly tackle long-horizon tasks such as software engineering, where a single trajectory may span dozens of tool-use steps. A recurring failure mode is *progress stagnation*: the agent enters a regime of surface-diverse but goal-unrelated actions—trying different commands, editing files, reading documentation—without making meaningful progress toward the objective. Unlike simple repetition loops, stagnation is semantically subtle: each individual action may appear reasonable, and only the *window-level* pattern reveals the lack of forward momentum.

Detecting stagnation in real time would enable early termination of futile trajectories, saving computational resources and enabling meta-controllers to intervene. However, three fundamental challenges impede reliable detection: (a) stagnation is inherently ambiguous at the boundary between exploration and futility (our human annotators achieve only κ=0.52); (b) agent trajectories violate the i.i.d. assumption underlying standard data programming Ratner et al. [2017], as labeling functions exhibit systematic temporal dependencies; (c) trajectory-level outcome information can leak into window-level labels, creating inflated performance estimates.

We formulate stagnation detection as window-level binary classification over k-step windows of agent trajectories, and construct a weak-supervision pipeline consisting of four labeling functions and a GRU-based neural probe. Our original hypothesis was that a temporal label model exploiting LF dynamics would yield better labels than standard aggregation, and that the trained probe would exceed the LF ensemble ("student exceeds teacher"). **Neither hypothesis holds.** Instead, we find that:

1. A temporal label model (HMM-based) achieves 0.770 gold-set F1, statistically indistinguishable from FlyingSquid's 0.772, despite successfully recovering leading/lagging LF structure. Correlation-aware aggregation already captures the relevant dependencies without explicit temporal modeling.

2. The probe achieves 0.72 gold-set F1, *below* the LF oracle (0.745) rather than above it. High auto-label performance (0.91) partially reflects overfitting to LF noise patterns rather than learning genuine progress signals.

3. LLM-as-judge with chain-of-thought reasoning achieves 0.855 gold-set F1, clearly outperforming the probe by 13.5pp. The probe's advantage is limited to cost (615× cheaper) and latency (667× faster).

4. Cross-agent transfer from SWE-agent to OpenHands degrades probe F1 from 0.91 to 0.61 (-29.4pp).

We reframe these negative results as contributions to the empirical understanding of weak supervision on sequential agent data: a stagnation taxonomy and window-level formulation as a task contribution (§3); systematic characterization of data programming failure modes (§5.5); and evidence that GRU temporal modeling provides robustness to label noise (§5.9).

2 Related Work

Agent monitoring and intervention. Cycle detectors such as CDCS+CDSA CDCS [2024] and SpecRA SpecRA [2024] identify structural repetition but miss semantically diverse stagnation. Self-reflection approaches Shinn et al. [2023], Yao et al. [2024] rely on the trapped model's own judgment. Process reward models Lightman et al. [2023], Wang et al. [2024b] score individual steps but require step-level outcome labels and lose window-level temporal dynamics. Value-function methods Le et al. [2022] need shaped rewards. Our work differs by framing progress monitoring as a weak-supervision problem at the window level.

Data programming and weak supervision. Snorkel Ratner et al. [2017] and its successors Fu et al. [2020], Shin et al. [2022] assume conditionally independent labeling functions. FlyingSquid Fu et al. [2020] relaxes this to model pairwise correlations via the triplet method. These approaches assume i.i.d. data points; we empirically characterize how this assumption fails on sequential agent trajectories with temporally structured LF outputs.

LLM-as-judge. Using LLMs as evaluators Zheng et al. [2024] provides high-quality assessments but at substantial cost. We compare against LLM-as-judge baselines to establish an upper bound on detection quality.

3 Task Formulation

3.1 Stagnation Taxonomy

We define *progress stagnation* as a contiguous window of agent actions that fails to advance the agent toward its declared objective, despite surface-level activity. Through analysis of 4,880 SWE-agent trajectories from SWE-bench Verified, we identify three stagnation subtypes: (1) *repetitive stagnation*—the agent repeats structurally similar commands; (2) *exploratory stagnation*—the agent tries diverse approaches without convergence; (3) *partial-progress stagnation*—some file edits occur but produce no functional improvement.

Metric	Value
Windows annotated	420
Fleiss κ	0.52
Unanimous agreement	45%
Majority agreement	68%
Auto MV vs. gold agreement	73%
Gold stagnation prevalence	38%

Table 1: Human annotation statistics. Moderate κ reflects inherent ambiguity of the progress/stagnation boundary.

Type	%	Pattern
Partial progress	43%	Edits but no test improvement
Exploration vs stag.	26%	Trying different approaches
Outcome-influenced	16%	Knowledge of final result
Annotation error	15%	Missed context / fatigue

Table 2: Breakdown of annotator disagreements (n=135 disagreement cases out of 420 windows).

3.2 Window-Level Formulation

Given a trajectory $\mathcal{T} = (a_1, a_2, \ldots, a_T)$ of agent actions, we extract overlapping windows $w_i = (a_i, a_{i+1}, \ldots, a_{i+k-1})$ of size k. Each window is labeled as PROGRESS or STAGNATION. We set k=5 based on ablation (Table 16), yielding 54,809 windows from 4,880 trajectories.

3.3 Labeling Functions

We design four labeling functions that capture complementary signals:

- **command_repetition**: Flags windows where $\geq$3 of 5 steps execute structurally similar commands. Leading indicator (fires $\sim$1.8 steps before stagnation onset).
- **output_similarity**: Measures pairwise cosine similarity of step outputs within the window. Concurrent indicator.
- **edit_cycle_detection**: Detects file-edit cycles (edit $\rightarrow$ revert $\rightarrow$ re-edit patterns). Concurrent indicator.
- **outcome_backpropagation**: Propagates trajectory-level outcome (resolved/unresolved) backward to label late windows. Lagging indicator (fires $\sim$2.1 steps after onset).

Labels are aggregated via majority vote. We define label *confidence* as the number of agreeing LFs: conf$\geq$2 yields 10,780 high-confidence windows; conf$\geq$1 yields 38,161 windows (72% at conf=1 with substantial LF disagreement).

3.4 Human Annotation (Gold Set)

Three annotators labeled 420 windows (4.2 min/window average; 29.4 hours total). Table 1 summarizes inter-annotator agreement.

Disagreement analysis (Table 2) reveals that 43% of annotator disagreements involve partial-progress cases, and 26% concern exploration-vs-stagnation distinctions. Notably, 16% of disagreements are influenced by annotator knowledge of the trajectory's final outcome, mirroring the `outcome_bp` leakage issue in our LFs.

4 Methods

4.1 Label Models

Majority Vote (MV). Standard unweighted vote across four LFs.

FlyingSquid. Correlation-aware label model Fu et al. [2020] using the triplet method to estimate pairwise LF dependencies without temporal structure.

Temporal Label Model. We propose an HMM-based label model that explicitly incorporates LF temporal roles. The model assigns each LF a temporal weight based on its lead/lag relative to stagnation onset, learned via EM on the LF output sequences. Two variants: (1) HMM with 2 hidden states; (2) weighted vote with learned temporal weights.

4.2 ProgressProbe

ProgressProbe is a lightweight GRU-based binary classifier. Each step's action text is encoded via Qwen3-Embedding-0.6B (1024d). A single-layer GRU (hidden size 128) processes the k-step embedding sequence, and a linear head classifies the final hidden state. Training uses cross-entropy loss on weak labels; we also evaluate a SupCon auxiliary loss variant. Total parameters: 198K (GRU + mean-pool) or 285K (GRU + attention).

4.3 Baselines

- **Repeat-3**: Flags stagnation if $\geq$3 consecutive steps share $>$90% command overlap.
- **CosineSim**: Thresholds mean pairwise cosine similarity within window (optimized threshold $=$ 0.78).
- **Mean-Pool + LR/MLP**: Logistic regression or 2-layer MLP on mean-pooled step embeddings (no temporal modeling).
- **Structural (GBT)**: Gradient-boosted trees on 6 non-NLP features (lines changed, file count, test count, command-type entropy, AST diff size, edit frequency).
- **LF Oracle**: Direct application of LF ensemble at inference time (majority vote on real-time LF outputs).
- **LLM-as-Judge**: Claude 3.5 Sonnet and GPT-4o in zero-shot, few-shot (3 examples), and chain-of-thought (CoT) settings.

5 Experiments

5.1 Main Results on Gold Set

Table 3 presents all methods evaluated on the 420-window human-annotated gold set.

Three findings stand out. First, LLM-as-judge with CoT clearly dominates in accuracy (Claude CoT: 0.855), establishing an upper bound that the probe (0.720) falls 13.5pp short of. Second, the probe does not exceed the LF oracle (0.745 vs. 0.720, $\Delta{=}{-}2.5$pp), invalidating the "student exceeds teacher" hypothesis. Third, the probe's practical advantage is limited to cost (615$\times$ cheaper than LLM-as-judge) and latency (667$\times$ faster), making it suitable only for high-volume continuous monitoring where LLM costs are prohibitive.

Method	Gold F1	Cost/win.	Latency
LLM-as-Judge			
Claude CoT	0.855	$0.042	3.8s
Claude few-shot	0.820	$0.035	2.9s
GPT-4o CoT	0.840	$0.038	3.5s
GPT-4o few-shot	0.808	$0.032	2.7s
Label Model / Oracle			
LF Oracle (MV)	0.745	$\sim$0	$\sim$0
FlyingSquid	0.772	$\sim$0	$\sim$0
Temporal LM (HMM)	0.770	$\sim$0	$\sim$0
Neural Probes			
ProgressProbe (GRU)	0.720	$5.2e-5	4.2ms
Mean-Pool MLP	0.701	$5.2e-5	1.8ms
Mean-Pool LR	0.688	$5.2e-5	0.5ms
Heuristic / Structural			
Structural (GBT)	0.680	$\sim$0	0.3ms
Repeat-3	—	$\sim$0	$\sim$0
CosineSim (opt.)	0.582	$\sim$0	0.8ms

Table 3: Main results on gold set. LLM-as-judge methods set the quality upper bound; the probe offers a 615$\times$ cost advantage at reduced accuracy. The probe does not exceed the LF oracle.

Seed	Auto F1	Gold F1	Δ Repeat-3	Δ LF Oracle
42	0.906	0.738	+6.2pp	$-$0.8pp
123	0.887	0.692	+4.4pp	$-$2.5pp
456	0.915	0.725	+7.2pp	$-$1.2pp
Mean$\pm$Std	0.903$\pm$.019	0.718$\pm$.031	+5.9$\pm$1.4	$-$1.5$\pm$0.9

Table 4: Multi-seed stability. Auto-label F1 is stable (std 0.019), but gold-set F1 shows greater variance (std 0.031) and consistently falls below the LF oracle.

5.2 Multi-Seed Stability

Table 4 shows probe performance across three random seeds, revealing the gap between auto-label and gold-set evaluation.

The divergence between auto-label performance (0.903) and gold-set performance (0.718) indicates that part of the probe's auto-label success reflects overfitting to LF noise patterns rather than learning true progress signals.

5.3 Label Model Comparison

Table 5 compares label aggregation strategies on the gold set. The temporal label model (HMM) achieves 0.770 gold-set F1, indistinguishable from FlyingSquid (0.772, Δ=$-$0.2pp). Both improve over majority vote by $\sim$2.5pp.

The HMM successfully learns leading/lagging weights (Table 6), but these adjustments flip only 8.3% of labels, with flipped-label accuracy on the gold set of approximately 55%—near random. FlyingSquid's correlation-aware aggregation already captures the LF dependencies without explicit temporal modeling, because the temporal structure manifests as statistical correlations that the triplet method recovers.

Label Model	Gold F1	Stag F1	Prog F1	Flip Rate
Majority Vote	0.745	0.680	0.810	—
FlyingSquid	0.772	0.712	0.832	11.2%
Temporal LM (HMM)	0.770	0.708	0.832	8.3%
Temporal LM (weighted)	0.762	0.698	0.826	6.5%

Table 5: Label model comparison. Temporal label models do not improve over FlyingSquid despite recovering temporal structure.

LF	Role	Lag (steps)	Weight
command_repetition	Leading	-1.8	1.35
output_similarity	Concurrent	0.0	1.12
edit_cycle_detection	Concurrent	-0.3	1.08
outcome_backprop.	Lagging	+2.1	0.72

Table 6: Learned temporal weights from the HMM label model. `command_repetition` is a leading indicator; `outcome_bp` is lagging.

5.4 LF Temporal Dynamics

Table 7 characterizes the temporal roles of each LF, confirming that LFs on agent trajectories violate the i.i.d. assumption of standard data programming.

`command_repetition` fires $\sim$1.8 steps before stagnation onset, while `outcome_bp` fires $\sim$2.1 steps after. However, high lag-1 autocorrelation (0.72–0.85) means LF outputs change slowly, and within a 5-step window, leading/lagging differences are averaged out. LF transition consistency is only 0.61—insufficient for reliable temporal label adjustment. This explains why the temporal label model gains no advantage over FlyingSquid: the temporal signal exists but is too weak at the window granularity to improve label aggregation.

5.5 Failure Mode: Outcome Leakage

Outcome backpropagation leakage. The `outcome_bp` LF propagates trajectory-level outcome information backward to window-level labels. Table 8 quantifies this leakage.

Exit context is present in 78% of trajectories, affecting 31% of windows. The `outcome_bp` marginal contribution (0.074) is the largest among all LFs (Table 9), yet it encodes "whether the task ultimately succeeded" rather than "whether this window makes progress." This leakage inflates the entire pipeline's performance: the "clean" performance (without `outcome_bp`) is 0.832 on auto-labels, and likely 0.68–0.70 on the gold set.

LF hold-one-out analysis. Table 9 shows marginal contributions of each LF. `outcome_bp` dominates the ensemble, contributing 7.4pp—more than double the next most important LF.

5.6 Failure Mode: Cross-Agent Transfer

We evaluate all methods on 2,340 OpenHands trajectories (18,720 windows) to test cross-implementation transfer from SWE-agent. Table 10 reveals severe degradation across all methods.

The probe's degradation (-29.4pp) exceeds even the LF oracle's (-26.0pp), directly contradicting the hypothesis that the probe learns agent-agnostic progress signals. LF coverage on OpenHands drops to 82% (vs. 95% on SWE-agent), with `outcome_bp` coverage declining most sharply (-16%). The LF oracle—despite also degrading—outperforms the probe on cross-agent evaluation by 7.3pp, suggesting that heuristic rules transfer more robustly than learned features.

LF	Lead/Lag	Role	Autocorr.	Onset Rate
cmd_repetition	-1.8	Leading	0.72	0.68
output_similarity	-0.5	Concurrent	0.65	0.61
edit_cycle_det.	-0.2	Concurrent	0.58	0.55
outcome_bp	+2.1	Lagging	0.85	0.78

Table 7: LF temporal dynamics. High autocorrelation ($>$0.58) indicates slow-varying LF outputs; the leading/lagging structure is systematic but diluted within k=5 windows.

Variant	Macro-F1	Stag F1	Prog F1	Δ
Original (all LFs)	0.906	0.859	0.952	—
Hold-out outcome_bp	0.832	0.762	0.902	-7.4pp
outcome_bp last-3 only	0.868	0.815	0.921	-3.8pp
outcome_bp last-1 only	0.812	0.738	0.886	-9.4pp

Table 8: Outcome backpropagation sensitivity. Holding out `outcome_bp` drops auto-label F1 by 7.4pp, with stagnation F1 most affected ($-$9.7pp).

5.7 LLM-as-Judge

Table 12 provides the full comparison. LLM-as-judge with CoT reasoning achieves substantially higher gold-set F1 than the probe, with Claude CoT reaching 0.855 ($+$13.5pp over probe).

At full scale (54K windows), LLM CoT evaluation costs $\sim$\$1,750 versus $\sim$\$2.80 for the probe. For continuous monitoring of millions of windows, LLM evaluation is economically infeasible, making the probe the only viable option despite its lower accuracy.

5.8 Error Analysis

We analyze 112 probe errors on the gold set (Table 13).

False negatives dominate (64 vs. 48 FPs). The largest FN category (33%) involves semantically diverse stagnation: the agent executes varied but collectively futile actions. These cases require goal-level reasoning to detect—exactly where LLM CoT excels and embedding-based probes fail. Among FPs, 73% fall in conf=1 regions (high LF disagreement), and 35% depend on `outcome_bp`, confirming that FPs are concentrated in the noisiest parts of the label space.

5.9 GRU Robustness to Label Noise

Table 14 compares training on high-confidence (conf$\geq$2) versus full (conf$\geq$1) data.

A notable reversal occurs: while auto-label F1 drops 5.9pp on noisy data (expected), gold-set F1 *increases* by 1.5pp. The larger training set compensates for noise, and the GRU's temporal modeling helps extract signal from noisy labels. This effect is amplified when comparing GRU vs. mean-pooling on noisy data: GRU gains +6.87pp over mean-pool MLP at conf$\geq$1 (Table 15), compared to +4.83pp at conf$\geq$2, confirming that temporal modeling provides disproportionate benefit under high label noise.

5.10 Ablations

Window size. Table 16 shows k=5 is optimal, though gold-set differences are small (1.5pp range).

Architecture. Table 17 compares architectures. Gold-set differences span only 2.7pp, suggesting label quality—not architecture—is the bottleneck.

Held-out LF	Auto F1	Δ	Marginal
None (full)	0.906	—	—
command_repetition	0.875	−3.1pp	0.031
output_similarity	0.882	−2.4pp	0.024
edit_cycle_detection	0.891	−1.5pp	0.015
outcome_backpropagation	0.832	−7.4pp	0.074

Table 9: LF hold-one-out. The leakiest LF (`outcome_bp`) is also the most influential—a cautionary pattern for weak supervision practitioners.

Method	Same-Agent	Cross-Agent	Degrad.
Probe (GRU)	0.906	0.612	−29.4pp
LF Oracle (MV)	0.945	0.685	−26.0pp
Repeat-3	0.843	0.590	−25.3pp
Mean-Pool LR	0.843	0.558	−28.5pp
CosineSim	0.582	0.480	−10.2pp

Table 10: Cross-agent transfer results. The probe degrades more than the LF oracle (−29.4 vs. −26.0pp), contradicting the hypothesis that it learns transferable progress signals.

Encoder. Four embedding encoders produce a 1.8pp range on the gold set (Table 18), confirming encoder choice is not the performance bottleneck.

SupCon auxiliary loss. Adding supervised contrastive loss yields +0.6pp on auto-labels and +0.3pp on the gold set—negligible. Embedding separation improves (0.74→0.82) but does not translate into meaningful classification gains when labels are noisy.

NL vs. structural features. NL embeddings outperform non-NLP structural features (GBT on 6 numeric features) by 4.0pp on the gold set (0.720 vs. 0.680), confirming that semantic content provides signal beyond structural patterns, though the gap is moderate.

Data scaling. Performance saturates at ∼50% of training data: gold-set F1 reaches 0.715 at 50% (5,400 windows) vs. 0.720 at 100%, indicating the bottleneck is label quality, not quantity.

CosineSim baseline. The original CosineSim baseline (F1=0.313) suffered from threshold miscalibration (default 0.5 in a high-similarity embedding space). Threshold sweep identifies optimal threshold=0.78, yielding corrected F1=0.582.

5.11 Early Termination Simulation

To assess downstream utility, we simulate probe-guided early termination of agent trajectories (Table 21).

A conservative threshold (t=0.8) saves 18.5% of tokens with only 1.7pp success-rate loss and 6.2% false termination rate. However, aggressive thresholds lead to unacceptable false termination rates (28.5% at t=0.4). The oracle shows that a perfect detector could save 35.2% of tokens with negligible cost, illustrating the theoretical value of stagnation detection—and the practical gap that current probe accuracy leaves.

5.12 Interpretability Analysis

t-SNE visualization of GRU hidden states shows moderate separation between progress and stagnation clusters (silhouette score 0.31), with substantial overlap at class boundaries. Attention weight analysis

LF	SWE-agent	OpenHands	Δ
command_repetition	0.95	0.88	−7%
output_similarity	0.92	0.85	−7%
outcome_bp	0.78	0.62	−16%
edit_cycle_detection	0.88	0.79	−9%
Ensemble	0.95	0.82	−13%

Table 11: LF coverage across agent implementations. `outcome_bp` suffers the largest drop.

Model	Setting	Gold F1	Stag	Prog	Cost
Claude 3.5	zero-shot	0.780	0.725	0.835	$0.028
Claude 3.5	few-shot	0.820	0.772	0.868	$0.035
Claude 3.5	CoT	0.855	0.810	0.900	$0.042
GPT-4o	zero-shot	0.765	0.710	0.820	$0.025
GPT-4o	few-shot	0.808	0.758	0.858	$0.032
GPT-4o	CoT	0.840	0.795	0.885	$0.038
Probe	—	0.720	0.650	0.790	$5.2e-5

Table 12: LLM-as-judge vs. probe on gold set. LLMs achieve higher accuracy via semantic understanding of goal-directed progress; the probe's advantage is purely economic.

reveals no clear "stagnation onset detection" pattern: weights distribute relatively uniformly across the window, with a slight bias toward the middle steps rather than concentrating on transition points. The GRU appears to perform coarse-grained pattern matching rather than fine-grained temporal reasoning, consistent with its moderate gold-set performance.

6 Discussion

What data programming gets wrong on sequential data. Our findings reveal a fundamental tension: LFs on agent trajectories exhibit systematic temporal dependencies (leading/lagging structure) that violate the i.i.d. assumption, yet explicitly modeling these dependencies (via temporal label model) yields no improvement over correlation-aware methods (FlyingSquid) that are agnostic to temporal order. The explanation is that within k=5 windows, temporal dynamics are averaged out—the signal exists at a finer granularity than the task formulation captures.

The leaky-heuristic trap. Our strongest LF (`outcome_bp`) is also the leakiest, contributing 7.4pp while encoding trajectory-level outcome rather than window-level progress. This creates a pernicious feedback loop: the most informative heuristic inflates both label quality and downstream model performance, making the pipeline appear more effective than it is. We estimate "clean" probe performance (without `outcome_bp` influence) at 0.68–0.70 on the gold set—near the structural baseline.

When probes are (not) useful. The probe's gold-set F1 (0.72) is clearly below LLM-as-judge CoT (0.855). Its value proposition is limited to scenarios where LLM evaluation is economically infeasible: continuous monitoring of >100K windows, where the 615× cost advantage dominates. For smaller-scale evaluation, LLM-as-judge is strictly preferable.

Inherent task ambiguity. Human annotator agreement (κ=0.52) sets a ceiling on how cleanly any method can separate progress from stagnation. The boundary is genuinely fuzzy—exploration can look like stagnation, and partial progress resists binary classification. Future work should consider ordinal or continuous progress scales rather than binary labels.

Error Type	N	%	Pattern
FN: Semantic-diverse stag.	37	33	Surface-diverse, goal-unrelated
FN: Partial progress	17	15	Mixed progress and stagnation
FN: Other	10	9	Edge cases
FP: Exploratory actions	25	22	Trial-and-error $\approx$ stagnation
FP: conf=1 region	35	31	LF disagreement zone

Table 13: Error breakdown. The dominant failure mode (33%) is semantically diverse stagnation, which requires goal-level reasoning beyond embedding statistics.

Training Set	N	Auto F1	Gold F1	Stag F1
conf$\geq$2	10,780	0.906	0.720	0.859
conf$\geq$1 (full)	38,161	0.847	0.735	0.777
Δ	+27,381	$-$5.9pp	+1.5pp	$-$8.2pp

Table 14: Label noise tolerance. The GRU trained on noisy full data (conf$\geq$1) achieves *higher* gold-set F1 than on clean conf$\geq$2 data, despite lower auto-label F1.

7 Conclusion

We present an empirical study of weak supervision for progress stagnation detection in LLM agent trajectories. Our original hypotheses—that temporal label modeling and neural probes would advance the state of detection—do not hold. Instead, we contribute: (1) a systematic characterization of data programming failure modes on sequential agent data, including i.i.d. violation through LF temporal dynamics and outcome leakage through backward-propagating heuristics; (2) a stagnation taxonomy and window-level formulation that provide a concrete task definition; (3) empirical evidence that GRU temporal modeling provides disproportionate robustness to label noise (+6.9pp over mean-pooling at high noise). These findings provide cautionary lessons and empirical foundations for future work on agent monitoring via weak supervision.

Limitations

Our study has several limitations. (1) All experiments use software engineering agent trajectories (SWE-bench); generalization to other domains (web navigation, embodied agents) is unknown. (2) Cross-agent transfer is evaluated on only two implementations (SWE-agent and OpenHands). (3) The gold set (420 windows) is small relative to the full dataset; larger annotation efforts could yield tighter performance estimates. (4) Human annotator agreement (κ=0.52) questions the reliability of the gold set itself as ground truth. (5) We do not evaluate online deployment or integration with meta-controllers.

References

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. In *Proc. VLDB*, 2017.

Setting	Pool LR	Pool MLP	GRU	GRU gain
Auto conf$\geq$2	0.844	0.857	0.906	+4.83pp
Auto conf$\geq$1	0.759	0.778	0.847	+6.87pp
Gold set	0.688	0.701	0.720	+1.9pp

Table 15: GRU vs. mean-pooling. Temporal modeling's advantage grows under noisier labels (conf$\geq$1).

k	N	Auto F1	Stag F1	Prog F1	Gold F1
3	82,214	0.878	0.812	0.944	0.705
5	54,809	0.906	0.859	0.952	0.720
7	39,149	0.895	0.848	0.942	0.718

Table 16: Window size ablation. k=5 balances signal accumulation and onset dilution.

Daniel Y. Fu, Mayee F. Chen, Frederic Sala, Sarah M. Hooper, Kayvon Fatahalian, and Christopher Ré. Fast and three-rious: Speeding up weak supervision with triplet methods. In *Proc. ICML*, 2020.

Jieyu Shin, Gilles Baechler, Weilin Xu, Ranjay Krishna, and Christopher Ré. Universalizing weak supervision. In *Proc. ICLR*, 2022.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *Proc. ICLR*, 2023.

Peiyi Wang, Lei Li, Zhihong Shao, R.X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proc. ACL*, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Proc. NeurIPS*, 2023.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Proc. NeurIPS*, 2024.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu-Hong Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. NeurIPS*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Proc. NeurIPS*, 2024.

Agent Cycle Detection Consortium. Detecting and mitigating repetitive cycles in LLM agent trajectories. *arXiv preprint*, 2024.

SpecRA Team. Spectral analysis for anomaly detection in agent trajectories. *arXiv preprint*, 2024.

Architecture	Params	Auto F1	Gold F1	ms
GRU + Attention	285K	0.912	0.728	5.1
GRU + Mean-pool	198K	0.906	0.720	4.2
Transformer (2L)	1.24M	0.903	0.722	8.7
MLP on pooled	132K	0.857	0.701	1.8

Table 17: Architecture ablation. Gold-set performance is insensitive to architecture choice (2.7pp range), confirming the bottleneck is label quality.

Encoder	Dim	Auto F1	Gold F1	min
Qwen3-Emb-0.6B	1024	0.906	0.720	53
BGE-M3	1024	0.898	0.715	48
E5-large-v2	1024	0.892	0.708	45
GTE-base	768	0.885	0.702	32

Table 18: Encoder ablation.

Train %	N	Auto F1	Gold F1
10%	$\sim$1,080	0.862	0.695
25%	$\sim$2,700	0.882	0.708
50%	$\sim$5,400	0.898	0.715
100%	$\sim$10,780	0.906	0.720

Table 19: Data scaling. Early saturation ($\sim$50%) confirms label quality as the bottleneck.

Threshold	Prec.	Recall	F1
0.50 (default)	0.218	0.875	0.313
0.70	0.412	0.723	0.498
0.78 (optimal)	0.561	0.608	0.582
0.85	0.645	0.391	0.472
0.90	0.712	0.198	0.295

Table 20: CosineSim threshold sweep. Even after optimization, CosineSim remains the weakest baseline, confirming stagnation $\neq$ surface repetition.

Threshold	Success Rate	Tokens Saved	False Term.
Baseline (no term.)	28.5%	0%	0%
Probe t=0.8	26.8%	18.5%	6.2%
Probe t=0.6	24.2%	32.1%	14.8%
Probe t=0.4	19.8%	48.7%	28.5%
Oracle	28.0%	35.2%	0.5%

Table 21: Early termination. Conservative threshold (t=0.8) saves 18.5% tokens with minimal success-rate impact ($-$1.7pp). The oracle gap highlights the cost of imperfect detection.